



HACKTHEBOX



Blacksmith

15th April 2022 / Document No. D22.102.239

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Easy**

Classification: Official

Synopsis

Blacksmith is a very easy difficulty challenge that features bypassing `seccomp` rules and reading the flag with open-read-write technique by crafting shellcode.

Skills Learned

- Seccomp rules shellcode crafting.

Enumeration

First of all, we start with `checksec` :

```
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX disabled
PIE:       PIE enabled
RWX:       Has RWX segments
```

Protections

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows
NX	✗	Disables code execution on stack

Protection	Enabled	Usage
PIE	✓	Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only
RWX	Has RWX segments	We can write and execute code

As we can see:

- PIE enabled
- Full RELRO
- Canary enabled
- But, NX is disabled!

That means there might be a possible code execution. No possible leak or overflow can occur due to the canary and pie.

`.got` overwrite is also impossible due to Full RELRO. We see that the binary has RWX segments. We will take advantage of this later on.

The interface of the program looks like this:

```

Traveler, I need some materials to fuse in order to create something
really powerful!
Do you have the materials I need to craft the Ultimate Weapon?
1. Yes, everything is here!
2. No, I did not manage to bring them all!
> 1
What do you want me to craft?
1. 🗡️
2. 🛡️
3. 🏹
> 2
Excellent choice! This luminous shield is empowered with Sun's light! *
It will protect you from any attack and it can reflect enemies attacks
back!
Do you like your new weapon?
> shellcode here
/home/ctf/run_challenge.sh: line 2: 30 Segmentation fault
./blacksmith
^C

```

We cannot understand a lot from this. Let's head into the disassembly.

Disassembly

Starting from `main()`:

```

void main(void)

{
    size_t __n;

```

```

long in_FS_OFFSET;
int local_28;
int local_24;
char *local_20;
char *local_18;
long local_10;

local_10 = *(long *) (in_FS_OFFSET + 0x28);
setup();
local_20 = "You are worthy to carry this Divine Weapon and bring peace to our
homeland!\n";
local_18 = "This in not a weapon! Do not try to mock me!\n";
puts("Traveler, I need some materials to fuse in order to create something
really powerful!");
printf(
    "Do you have the materials I need to craft the Ultimate Weapon?\n1.
Yes, everything ishere!\n2. No, I did not manage to bring them all!\n> "
);
__isoc99_scanf(&DAT_00101299,&local_28);
if (local_28 != 1) {
    puts("Farewell traveler! Come back when you have all the materials!");
    /* WARNING: Subroutine does not return */
    exit(0x22);
}
printf("What do you want me to craft?\n1. ⚔️\n2. ❤️\n3. 🛡️\n> ");
__isoc99_scanf(&DAT_00101299,&local_24);
sec();
if (local_24 == 2) {
    shield();
}
else {
    if (local_24 == 3) {
        bow();
    }
    else {
        if (local_24 != 1) {
            __n = strlen(local_18);
            write(1,local_18,__n);
            /* WARNING: Subroutine does not return */
            exit(0x105);
        }
        sword();
    }
}
if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

There are many functions that are called:

- setup()
- sec()

- shield()
- sword()
- bow()

Seccomp rules

Starting with sec():

```
void sec(void)

{
    long lVar1;
    undefined8 uVar2;
    long in_FS_OFFSET;

    lVar1 = *(long *) (in_FS_OFFSET + 0x28);
    prctl(0x26,1);
    prctl(4,0);
    uVar2 = seccomp_init(0);
    seccomp_rule_add(uVar2,0x7fff0000,2,0);
    seccomp_rule_add(uVar2,0x7fff0000,0,0);
    seccomp_rule_add(uVar2,0x7fff0000,1,0);
    seccomp_rule_add(uVar2,0x7fff0000,0x3c,0);
    seccomp_load(uVar2);
    if (lVar1 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}
```

We see `seccomp` rules. To get a better understanding:

```
w3th4nds@void:~/github/HTB-Challenges/pwn/blacksmith-
seccomp_open_read_write_easy/challenge$ sudo seccomp-tools dump ./blacksmith
[sudo] password for w3th4nds:
Traveler, I need some materials to fuse in order to create something really
powerful!
Do you have the materials I need to craft the Ultimate Weapon?
1. Yes, everything is here!
2. No, I did not manage to bring them all!
> 1
What do you want me to craft?
1. 🗡️
2. 🛡️
3. 🏹
> 2
line  CODE  JT   JF      K
=====
0000: 0x20 0x00 0x00 0x00000004  A = arch
0001: 0x15 0x00 0x08 0xc000003e  if (A != ARCH_X86_64) goto 0010
0002: 0x20 0x00 0x00 0x00000000  A = sys_number
0003: 0x35 0x00 0x01 0x40000000  if (A < 0x40000000) goto 0005
0004: 0x15 0x00 0x05 0xffffffff  if (A != 0xffffffff) goto 0010
```

```

0005: 0x15 0x03 0x00 0x00000000  if (A == read) goto 0009
0006: 0x15 0x02 0x00 0x00000001  if (A == write) goto 0009
0007: 0x15 0x01 0x00 0x00000002  if (A == open) goto 0009
0008: 0x15 0x00 0x01 0x00000003c  if (A != exit) goto 0010
0009: 0x06 0x00 0x00 0x7fff0000  return ALLOW
0010: 0x06 0x00 0x00 0x00000000  return KILL

```

The only system functions that are allowed are:

- open
- read
- write
- exit

That means we cannot insert `execve("/bin/sh")` or something similar to get shell because the process will be killed.

Open - Read - Write

`shield()` function looks like this:

```

void shield(void)

{
    size_t __n;
    long in_FS_OFFSET;
    undefined local_58 [72];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    __n = strlen(
        "Excellent choice! This luminous shield is empowered with Sun\'s
        light! *\nIt will protect you from any attack and it can reflect enemy attacks
        back!\n"
    );
    write(1,&DAT_00101080,__n);
    __n = strlen("Do you like your new weapon?\n> ");
    write(1,"Do you like your new weapon?\n> ",__n);
    read(0,local_58,0x3f);
    (*(code *)local_58)();
    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}

```

As we can see, there is code execution at: `(*(code *)local_28)();`

We can store whatever we want in this buffer and it will be executed! "Whatever we want" means `open-read-write` otherwise it will be killed. Luckily enough, `pwn tools` can craft a shellcode for us.

Also, the description said something about `./flag.txt`, which means the flag is stored in the current directory.

Our goal is to:

- `open` `flag.txt`.
- `read` its content.
- `write` it on the screen.
- `exit`.

Solution



```
w3th4nds@void:~/github/HTB-Challenges/pwn/blacksmith-  
seccomp_open_read_write_easy/challenge$ ./solver.py  
[+] Opening connection to 172.17.0.1 on port 1312: Done  
[*] Flag: HTB{f4k3_fl4g_4_t35t1ng}
```